Universidade do Minho

AGENTIF AI

Síntese De Voz Em Tempo Real

Projeto de Informática



Beatriz Monteiro, Daniel Du, Daniel Furtado, Moisés Antunes, Pedro Gomes e Telmo Maciel

Grupo E04 - Agentifai

Braga, 29 de novembro de 2024

Resumo

No âmbito da Unidade Curricular Projeto de Informática, do segundo ano do Mestrado em Engenharia Informática, e em colaboração com a empresa Agentifai, foi desenvolvido um software destinado à síntese de voz em tempo real. O objetivo deste projeto consiste em criar um sistema capaz de gerar voz a partir de texto. A solução permitirá a personalização da voz e funcionará de forma independente de terceiros. Este sistema poderá ser integrado noutros sistemas. O presente relatório tem como finalidade documentar todas as etapas deste projeto, começando pela fase de levantamento de requisitos, passando pela implementação do sistema e culminando na elaboração de um modelo de negócio para o produto desenvolvido.

Conteúdo

1 Introdução 4
1.1 Contextualização 4
1.2 Motivação 5
1.3 Objetivos 5
2 Metodologia e Gestão de Trabalho 6
3 Levantamento de Requisitos 6
3.1 Técnicas de Levantamento de Requisitos 6
3.2 Restrições do Projeto 7
3.3 Requisitos Funcionais 7
3.4 Requisitos Não Funcionais 8
3.5 Use Cases 10
4 Desenvolvimento do Projeto 11
4.1 Arquitetura 11
4.1.1 Proxy 12
4.1.2 Modelo 12
4.1.3 Protocolo de *Streaming* 13
4.1.4 Base de Dados 14
5 Testes e Resultados 15
5.1 Resultados Obtidos 15
5.2 Comparações e Conclusões 16
6 Modelo de Negócios 17
7 Conclusão 17
7.1 Desafios 18
7.2 Trabalho Futuro 18
7.3 Agradecimentos 18
A Anexo 19
Anexo 1 Estrutura do Docker 19
Anexo 2 Análise do Mercado 21

Primeira Componente do Projeto

1 Introdução

1.1 Contextualização

Nos últimos anos os avanços das tecnologias tem permitido desenvolver soluções inovadoras em diversas áreas, incluindo na síntese de voz, que tem ganho relevância no contexto de interações homem-máquina, especialmente em assistentes virtuais e sistemas de atendimento ao cliente. O projeto **Síntese de Voz em Tempo Real** surge para atender à crescente procura por vozes mais naturais e personalizadas em aplicações comerciais.

Este projeto foi desenvolvido na unidade curricular de Projeto de Informática e realizado em parceria com a Agentifai, uma empresa especializada em soluções de inteligência artificial voltadas para o atendimento ao cliente. A proposta central do projeto envolve a criação de um sistema de síntese de voz que funcione em tempo real, garantindo uma resposta ágil e natural em interações com o utilizador. A Agentifai considera a síntese de voz uma ferramenta poderosa para melhorar a qualidade das interações nos seus sistemas de assistentes virtuais, oferecendo um atendimento mais eficiente e próximo da experiência humana.

No entanto, a implementação de sistemas de síntese de voz ainda apresenta desafios significativos, destacando-se os seguintes:

1. **Latência na Síntese:** A capacidade de sintetizar voz em tempo real é essencial para garantir uma comunicação fluida, especialmente em aplicações sensíveis ao tempo, como assistentes virtuais.
2. **Personalização:** A criação de vozes únicas a partir de amostras específicas exige um processamento sofisticado, que permita não apenas a clonagem, mas também a adaptação das vozes às preferências dos utilizadores.
3. **Integração com sistemas existentes:** É crucial que a solução desenvolvida funcione de forma independente, mas também se integre adequadamente com os assistentes virtuais da Agentifai, sem comprometer a funcionalidade ou a qualidade.

A execução do projeto requereu o uso de técnicas de Processamento de Linguagem Natural, incluindo redes neuronais para processamento e geração de voz natural, assim como a otimização do sistema para garantir baixa latência, permitindo uma experiência de resposta em tempo real. Além disso, foi dada especial atenção à naturalidade da voz gerada e à capacidade de adequação a diferentes contextos, visando atender a uma ampla gama de aplicações.

1.2 Motivação

A motivação para o desenvolvimento do projeto de síntese de voz em tempo real pela Agentifai baseia-se em quatro pilares: **redução de custos, personalização, naturalidade e velocidade, e flexibilidade de integração**.

1. A empresa procura **diminuir as despesas** associadas a APIs de voz de grandes fornecedores, como *Google*, visando maior autonomia e economia a longo prazo.
2. A **personalização das vozes** é crucial, pois a voz de um assistente virtual reflete a identidade de uma empresa, e é necessário que cada cliente tenha uma voz personalizada.
3. A **naturalidade e a rapidez da síntese de voz** são essenciais para melhorar a experiência do utilizador.
4. A **flexibilidade de integração** garante que o sistema possa ser usado em diversos canais de comunicação.

1.3 Objetivos

Os principais objetivos deste projeto são:

1. **Geração de voz a partir de texto:** Criar uma solução que sintetize voz natural a partir de texto, utilizando tanto vozes personalizáveis quanto a transformação de áudios gerados por outros sistemas de TTS(*Text-to--Speech*).
2. **Síntese de Voz em Tempo Real:** Permitir a geração de voz em tempo real a partir de texto ou áudio previamente gerado.
3. **Síntese em *streaming*:** Implementar a síntese de voz de forma a garantir a mínima latência, assegurando uma experiência de uso fluida e responsiva.
4. **Personalização da voz:** Permitir que os utilizadores desenvolvam vozes personalizadas a partir de uma coleção de amostras, criando perfis vocais únicos que atendam às suas necessidades.
5. **Integração noutros sistemas:** Garantir que a solução se integre facilmente outros sistemas, funcionando como uma funcionalidade adicional que melhore a experiência do utilizador.

Este projeto visa não apenas inovar na tecnologia de síntese de voz, mas também oferecer uma ferramenta que melhore a interação entre utilizadores e sistemas automatizados, contribuindo para um futuro mais dinâmico e personalizado.

2 Metodologia e Gestão de Trabalho

Para garantir o progresso estável do projeto foi necessário, em primeiro lugar, estabelecer uma comunicação constante entre as partes nele envolvidas. Assim que foi feito o grupo de trabalho, foi formado um servidor no Discord de modo a permitir troca de mensagens entre os seus membros a qualquer momento. Após a distribuição dos temas pelos grupos, foi estabelecido contacto com a Agentifai por email e, depois de uma reunião presencial na sede da empresa em Braga, foi decidido que haveria reuniões semanais às terças-feiras no final da manhã, em regime remoto através do uso do Google Meet, nas quais foram tiradas dúvidas e trocadas sugestões acerca do desenvolvimento do projeto com um representante da empresa.

As primeiras reuniões serviram para estabelecer as tarefas a fazer de acordo com os objetivos e os requisitos que nos foram propostos. Para facilitar as divisões de tarefas que seriam necessárias à medida que fomos avançando no trabalho, o projeto foi dividido numa vertente mais prática, correspondente ao desenvolvimento, e numa vertente mais conceptual, que consistia em outros aspetos como o levantamento de requisitos, o estabelecimento de um modelo de negócios e a redação deste relatório. Cada um dos membros esteve envolvido sobretudo numa vertente específica, mas houve alguma fluidez de modo a garantir que nenhuma tarefa ficaria demasiado atrasada.

Foi depois criado um espaço de trabalho no *Notion*, que permite registar tarefas a fazer, apontar notas, verificar dados de cada membro do grupo que possam ser importantes, como o email e número de aluno de cada um, bem como as respetivas responsabilidades, e guardar *links* para *sites* ou ficheiros que sejam importantes para a realização deste projeto, tais como o documento de requisitos, alguns *sites* com informação necessária no desenvolvimento do trabalho ou até mesmo este relatório.

3 Levantamento de Requisitos

3.1 Técnicas de Levantamento de Requisitos

O levantamento de requisitos para o sistema de TTS baseou-se em três abordagens principais:

- **Entrevistas e Reuniões:** Realizaram-se reuniões com João Cunha, responsável pelo projeto por parte da Agentifai, para recolher informações críticas sobre as necessidades e expectativas do sistema. Discutiram-se funcionalidades essenciais, qualidade da voz gerada, perceção de emoções, e desafios de integração com os assistentes virtuais da Agentifai.

- **Pesquisa de Mercado e Tecnológica:** Foram analisadas ferramentas de TTS (como Google Cloud TTS, Microsoft Azure Speech e Mozilla TTS). Esta pesquisa ajudou a identificar boas práticas, funcionalidades relevantes e o impacto da voz na experiência do utilizador.
- **Sessões de Brainstorming:** O grupo realizou sessões para explorar cenários de uso e refinar os requisitos. Estas sessões geraram ideias inovadoras e ajudaram a alinhar as soluções técnicas com as expectativas do cliente.

Estas estratégias garantiram uma definição sólida dos requisitos, alinhando o desenvolvimento do sistema TTS com as necessidades do projeto e as melhores práticas da indústria. O sistema foi projetado para ser robusto, flexível e preparado para integração com assistentes virtuais e outros sistemas futuros.

3.2 Restrições do Projeto

As restrições impostas, classificadas como à solução e temporais, são fundamentais para garantir que o produto final atenda às expectativas do cliente e às necessidades dos utilizadores, além de orientar o planeamento do projeto.

1. **Restrições à Solução:**
   - **Restrição de Integração:** O sistema TTS deverá ser implementado como uma funcionalidade dos assistentes virtuais existentes, assegurando compatibilidade e aproveitando o ecossistema atual da empresa. **Prioridade:** *Must*
   - **Restrição Técnica:** O sistema deve operar em infraestrutura própria, garantindo autonomia e eliminando dependências de terceiros. **Prioridade:** *Must*
2. **Restrições Temporais:**

O projeto deverá ser concluído e entregue até **29 de novembro de 2024**, conforme definido pela equipa docente.

A identificação e consideração destas restrições são cruciais para um planeamento eficaz e uma execução bem-sucedida, garantindo que o sistema TTS seja entregue nos prazos estabelecidos e atenda às necessidades técnicas e de integração exigidas.

3.3 Requisitos Funcionais

Os requisitos funcionais detalham as capacidades esperadas do sistema TTS, com base nas necessidades do cliente e no contexto do projeto. Estes

requisitos garantem que o sistema seja eficaz, flexível e capaz de atender às expectativas de utilização.

1. **Requisitos Essenciais (Prioridade: *Must*):**
   - **Requisito 1** Produzir áudio a partir de texto de entrada, permitindo que os utilizadores ouçam as respostas geradas.
   - **Requisito 2** Permitir a integração com assistentes virtuais já existentes, assegurando sinergia com a infraestrutura atual e uma experiência coesa.
   - **Requisito 3** Suporte ao idioma português, garantindo uma interação eficiente e natural com os utilizadores.
2. **Requisitos Relevantes (Prioridade: *Should*):**
   - **Requisito 4** Oferecer opções de vozes pré-configuradas, permitindo aos consumidores personalizarem a experiência de uso.
3. **Requisitos Desejáveis (Prioridade: *Could*):**
   - **Requisito 5** Adaptar a entoação da voz com base em palavras-chave, para melhorar a compreensão da informação transmitida.
   - **Requisito 6** Ajustar a emoção na voz gerada conforme o contexto da conversa, promovendo interações mais naturais e envolventes.
   - **Requisito 7** Processar múltiplos formatos de texto, como HTML, Markdown, e-mails e texto simples, para aumentar a flexibilidade e integração.

Os requisitos funcionais estabelecem uma base clara para o desenvolvimento do sistema TTS, priorizando funcionalidades críticas enquanto exploram possibilidades de personalização e interação natural. Este conjunto de funcionalidades reflete as necessidades do cliente e as melhores práticas para garantir uma solução robusta e versátil.

3.4 Requisitos Não Funcionais

Os requisitos não funcionais garantem que o sistema TTS seja robusto, escalável, seguro e preparado para atender às exigências operacionais e legais. São agrupados em categorias principais, cada uma refletindo uma dimensão crítica do desenvolvimento.

1. **Usabilidade:**
   - **RNF1** O sistema deve ser intuitivo e simples de usar para qualquer tipo de utilizador, independentemente do seu nível de conhecimento técnico. **Prioridade:** *Must*
2. **Performance:**

- **RNF2** O sistema deve responder à entrada de texto dentro de um limite de tempo de 1,5 segundos, assegurando fluidez na interação. **Prioridade:** *Must*
- **RNF3** O sistema deve estar preparado para suportar múltiplos utilizadores simultâneos sem degradação do desempenho. **Prioridade:** *Should*
- **RNF4** O sistema deve ser capaz de gerar uma voz clara. **Prioridade:** *Must*
- **RNF5** O sistema deve gerar uma voz com entoação natural, evitando sons robóticas e artificiais. **Prioridade:** *Must*
- **RNF6** O sistema deve estar disponível pelo menos 99% do tempo. **Prioridade:** *Must*

3. **Operacional:**
   - **RNF7** O sistema deve ser compatível com vários canais de comunicação. **Prioridade:** *Should*
   - **RNF8** O sistema deve operar como um *SaaS*. **Prioridade:** *Must*
4. **Manutenção e Suporte:**
   - **RNF9** O sistema deve implementar mecanismos de redundância para garantir a continuidade do serviço, mesmo em caso de falhas. **Prioridade:***Should*
   - **RNF10** O sistema deve ser fácil de manter, com código modular e bem documentado, para facilitar futuras atualizações ou correções. **Prioridade:***Must*
5. **Cultural e Político:**
   - **RNF11** O sistema deve suportar múltiplos idiomas. **Prioridade:***Should*
6. **Legal:**
   - **RNF12** As vozes utilizadas no sistema devem ser provenientes de fontes que autorizam a sua utilização, respeitando todas as licenças e direitos de autor. **Prioridade:** *Must*

Estes requisitos não funcionais asseguram que o sistema seja não apenas funcional, mas também eficiente, confiável, e alinhado às melhores práticas da indústria. Estabelecem padrões técnicos e operacionais necessários para alcançar uma solução de alta qualidade e preparada para o futuro.

3.5 Use Cases

O sistema TTS a ser desenvolvido tem os seguintes atores:

- **Sistema TTS** O próprio sistema de síntese de texto para fala (TTS), responsável por processar texto e gerar áudio com base nas funcionalidades configuradas.
- **Consumidor** O administrador da empresa que contrata os assistentes virtuais. Embora seja um ator secundário, desempenha um papel fundamental ao iniciar o processo de personalização da voz do assistente virtual, sendo que a execução prática é realizada pelo técnico.
- **Técnico** Profissional responsável por realizar manutenções, atualizações, e ajustes técnicos, como adicionar novas vozes e escolher a voz do assistente virtual.
- **Utilizador Final** A pessoa que interage indiretamente com o sistema TTS através de um assistente virtual, ouvindo as respostas em áudio geradas pelo sistema. Este é considerado um **ator secundário**.
- **Assistente Virtual** Funciona como a interface de comunicação entre o utilizador final e o sistema TTS. Atua como intermediário, processando as solicitações recebidas e enviando o texto correspondente ao sistema TTS para conversão em áudio. Posteriormente, é responsável por transmitir o áudio gerado ao utilizador, garantindo uma interação fluida e eficiente.

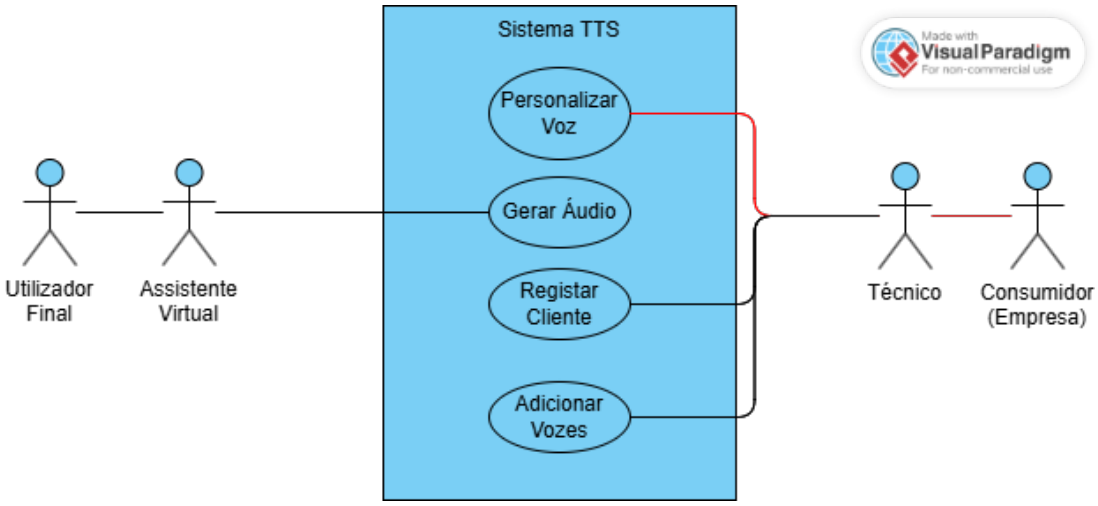


Figura 1: Diagrama de Use Cases*

* O Consumidor desencadeia a ação Personalizar Voz, realizada pelo Técnico(Linha Vermelha).

4 Desenvolvimento do Projeto

4.1 Arquitetura

Para atingir uma implementação modular, escalável e de fácil manutenção, foi adotada uma arquitetura baseada em micro-serviços. Esta abordagem permite que cada componente do sistema funcione de forma independente, facilitando a atualização, teste e implantação de cada serviço sem impactar os restantes. Além disso, possibilita a escalabilidade horizontal, permitindo que serviços específicos sejam replicados conforme o necessário.

Cada componente principal do projeto—*Server*, *Client*, *API*, *Proxy*, *Front-end* e *Normalizer* —foi encapsulado num *container Docker* individual. O uso do *Docker* assegura que cada micro-serviço tenha o seu próprio ambiente de execução isolado com as dependências necessárias. Isso não apenas elimina conflitos entre dependências de diferentes serviços, mas também garante que o sistema seja portátil e reproduzível em qualquer sistema que suporte *Docker*.

A explicação detalhada sobre a arquitetura e a interação entre os *containers Docker* está disponível na **Seção Anexo 1**.

Na figura abaixo está representada a interação entre os diferentes *containers*:

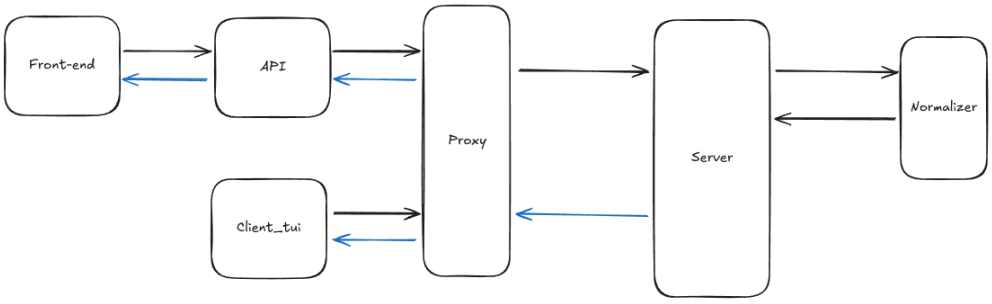


Figura 2: Arquitetura dos Docker *Containers*

As interações entre os *containers* foram desenhadas de forma modular e clara. As setas pretas representam o envio de texto, enquanto as setas azuis representam o envio de áudio.

A utilização de micro-serviços e *containers Docker* traz diversos benefícios:

- **Modularidade e Isolamento:** Cada serviço é independente, facilitando o desenvolvimento e a manutenção. Problemas num serviço não afetam diretamente os outros, aumentando a resiliência do sistema.
- **Flexibilidade Tecnológica:** Permite o uso de diferentes tecnologias e linguagens de programação em diferentes serviços, escolhendo a melhor ferramenta para cada tarefa específica.

- ***Continuous Integration and Continuous Delivery/Deployment* (CI/CD):** Facilita a implementação de *pipelines* de CI/CD, acelerando o ciclo de desenvolvimento e implantação de novas funcionalidades ou correções.
- **Portabilidade:** Os *containers* Docker garantem que o ambiente de execução seja consistente em diferentes plataformas, simplificando a implantação em diferentes infraestruturas, seja localmente ou na nuvem.

4.1.1 Proxy

O Proxy desempenha um papel central na arquitetura, atuando como um intermediário inteligente que gere a comunicação entre os clientes e múltiplas instâncias de servidores. Ele é responsável por:

- **Escalabilidade dinâmica:** Facilita a adição ou remoção de instâncias de servidores conforme preciso, sem a necessidade de reconfigurações complexas na arquitetura, garantindo que o sistema possa lidar eficientemente com variações no volume de solicitações.
- **Gestão de falhas**: Monitoriza a saúde dos servidores e redireciona automaticamente as solicitações em caso de falha de algum deles, aumentando a robustez e a disponibilidade do sistema.

4.1.2 Modelo

Para a implementação do modelo de síntese de voz, foi escolhida a biblioteca **Coqui TTS**, uma solução opensource que se destaca pela sua abrangência e flexibilidade. O Coqui TTS é mais completo em comparação com outras bibliotecas, pois integra uma variedade maior de modelos pré-treinados, como por exemplo o *Tacotron2* e o *FastSpeech2*. Esta diversidade de modelos permite ajustes mais precisos e a possibilidade de alcançar resultados de maior qualidade na conversão de texto para voz.

Adicionalmente, foi utilizado o *XTTS*, um modelo de geração de voz baseado nas tecnologias do Coqui TTS, que oferece a funcionalidade avançada de clonagem de vozes, inclusive para outras línguas. Esta capacidade é particularmente útil, pois permite replicar características específicas de uma voz, economizando tempo no processo de treino e facilitando a adaptação do modelo a diferentes necessidades linguísticas e vocais.

Para termos um modelo eficaz na conversão de texto para voz é necessário um treino meticuloso utilizando datasets de voz adequados. Os datasets de voz são conjuntos extensos de ficheiros de áudio que contêm, na medida do possível, o maior número de fonemas de uma ou mais línguas específicas, otimizando o espaço de memória utilizado. A diversidade fonética presente nestes datasets é crucial para garantir que o modelo possa reproduzir a fala de forma natural e precisa em diferentes contextos linguísticos.

Uma fonte comum para a obtenção de datasets de alta qualidade são os audiobooks. No entanto, embora forneçam uma abundância de fonemas e variações nos tons de voz e tipos de discurso, a utilização de audiobooks pode introduzir uma entoação narrativa específica que é indesejada para certos cenários. Esta entoação pode fazer com que o modelo reproduza a fala com um estilo narrativo, o que não é apropriado quando se pretende uma voz mais neutra ou adaptada a diferentes contextos. Para contornar este problema, é importante complementar os audiobooks com outros tipos de datasets que proporcionem uma entoação mais neutra. Isto pode incluir gravações de fala espontânea, diálogos ou frases isoladas, que ajudem o modelo a aprender diferentes padrões de entoação e prosódia sem o viés narrativo. Em resumo, embora os audiobooks forneçam um rico conjunto de dados fonéticos, a sua utilização isolada pode introduzir uma entoação narrativa indesejada. A combinação de diferentes tipos de datasets, juntamente com o uso de bibliotecas avançadas como o Coqui TTS e o XTTS, e a utilização dos recursos computacionais do Google Colab Pro, resultou num modelo de síntese de voz eficiente e versátil. Este modelo é capaz de reproduzir vozes com alta fidelidade e entoação adequada, oferecendo aplicações promissoras em diversas áreas que requerem conversão de texto para voz.

Por último, para o processo de treino, recorremos ao Google Colab Pro, que nos permitiu escrever e executar código Python sem a necessidade de configurações complexas. Para além disso, oferece-nos acesso às GPUs da Google permitindo acelerar significativamente o processo de treino dos modelos. Este ambiente computacional robusto é essencial para lidar com as exigências intensivas de processamento envolvidas no treino de modelos de síntese de voz de alta qualidade.

4.1.3 Protocolo de *Streaming*

No desenvolvimento do sistema, utilizámos a tecnologia *gRPC* (*Google Remote Procedure Call*) para a comunicação entre os diferentes componentes distribuídos da aplicação. O *gRPC* é uma *framework* de comunicação moderna e de alto desempenho que utiliza *Protocol Buffers* (*protobuf*) como linguagem de definição de interface e oferece suporte para múltiplas linguagens de programação.

A definição dos serviços *gRPC* foi realizada no ficheiro `tts.proto`, utilizando a sintaxe do Protocol Buffers (*proto3*). O serviço principal, ***TTSService***, inclui a síntese de texto, gestão de utilizadores e vozes. O método ***SynthesizeStream*** permite *streaming* bidirecional, enviando texto e recebendo áudio em fragmentos.

A geração de código foi feita com a ferramenta `grpcio-tools`, criando os ficheiros Python necessários. O ficheiro gerado `tts_pb2_grpc.py` contém duas classes principais:

- ***TTSServiceStub*** (cliente *gRPC*) para chamar os serviços remotos.
- ***TTSServiceServicer*** (servidor *gRPC*) que define os métodos do serviço, inicialmente lançando exceções para implementação gradual.

No servidor, os métodos são estruturados para lidar com diferentes tipos de RPC:

- ***SynthesizeStream*** processa streams bidirecionais de áudio.
- Métodos como ***AddUser*** e ***RemoveUser*** gerem utilizadores e vozes, retornando o *status* das operações.

O serviço de normalização de texto, definido em `normalizer.proto`, inclui o método *Normalize*, que retorna o texto normalizado. A comunicação eficiente entre os serviços de conversão de texto em áudio e normalização é garantida pelo uso de *gRPC*, que proporciona uma arquitetura modular, escalável e de alto desempenho.

4.1.4 Base de Dados

A base de dados foi implementada para armazenar informações sobre utilizadores (empresas clientes) e vozes do sistema TTS, facilitando a gestão de dados no contexto de assistentes virtuais personalizados. Utilizámos o *framework* ***SQLAlchemy*** para abstrair a interação com a base de dados, permitindo operações como inserções, consultas e associações de forma eficiente e segura. Foi escolhido o *SQLAlchemy* devido à sua flexibilidade e capacidade de abstrair operações complexas, além de permitir um desenvolvimento mais ágil e estruturado.

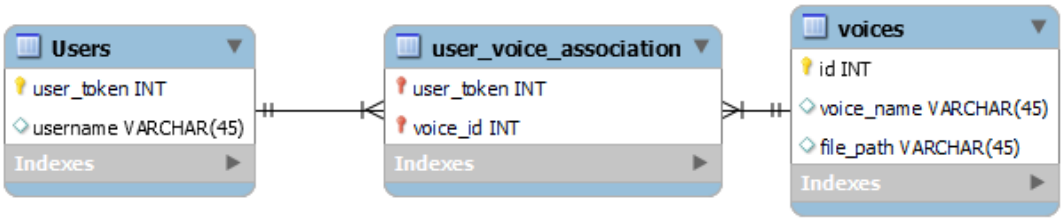


Figura 3: Modelo Lógico

A base de dados é estruturada com duas tabelas principais: uma para **utilizadores** e outra para **vozes**. Estas tabelas estão conectadas por uma tabela associativa que implementa um relacionamento muitos-para-muitos entre os clientes e as vozes, permitindo que vários clientes possam ter várias vozes associadas aos seus serviços.

Os modelos que representam as tabelas foram definidos em *Python*, e a estrutura da base de dados foi gerada automaticamente com *SQLAlchemy*. Foram adicionados dados iniciais para fins de teste, incluindo registos de utilizadores, vozes e as suas associações.

O *design* da base de dados garante escalabilidade e flexibilidade, permitindo personalização e gestão eficiente no contexto do sistema TTS. Esta implementação assegura a consistência e acessibilidade dos dados para futuras utilizações.

5 Testes e Resultados

No âmbito deste projeto, para além do desenvolvimento do sistema de síntese de voz, foi realizada uma análise empírica da qualidade dos modelos utilizados. Este estudo envolveu a realização de testes utilizando métricas objetivas e subjetivas, com o objetivo de avaliar a qualidade da voz gerada pelo modelo treinado.

As métricas selecionadas para esta análise incluem o *Short-Time Objective Intelligibility* (**STOI**), que mede a inteligibilidade do áudio gerado, e o *Signal-to-Noise Ratio* (**SNR**), que avalia a relação entre o sinal de fala e o ruído presente no áudio. Além disso, foi aplicada a métrica subjetiva *Mean Opinion Score* (**MOS**), por meio de inquéritos a utilizadores, para capturar a percepção subjetiva sobre a naturalidade e clareza da fala.

5.1 Resultados Obtidos

1. **Inteligibilidade da fala (STOI):** O cálculo do STOI comparou o áudio gerado pelo *Google Text-to-Speech* (`original.wav`) com o gerado pelo nosso modelo (`gerado.wav`). O valor obtido foi 0.20, o que indica uma inteligibilidade relativamente baixa na síntese do modelo treinado. No entanto, é importante notar que esse valor de STOI pode não refletir totalmente a qualidade percebida do áudio, pois o modelo treinado visa gerar uma voz mais natural e fluida, o que pode ser mais difícil de ser avaliado objetivamente por métricas como o STOI. Em comparação, o áudio do *Google Text-to-Speech* apresenta uma maior clareza acústica, mas de forma mais "artificial". Embora o valor do STOI seja baixo, a inteligibilidade do áudio gerado pelo modelo treinado é geralmente considerada boa por ouvintes humanos, como refletido em outras métricas subjetivas, como o MOS (*Mean Opinion Score*).
2. **Relação Sinal-Ruído (SNR):** Os testes demonstraram um SNR de 2.99 dB ao comparar o áudio gerado pelo modelo treinado (`gerado.wav`) com o áudio de referência (`original.wav`). Este valor sugere que o modelo introduz um nível considerável de ruído durante a síntese, o que resulta numa

relação sinal-ruído mais baixa em comparação com o *Google Text-to--Speech*. Embora o SNR obtido seja relativamente baixo, é importante notar que ainda está dentro de uma faixa aceitável para algumas aplicações práticas. Contudo, o valor sugere que há uma necessidade de melhorias na redução de ruídos residuais para alcançar uma qualidade sonora mais próxima ao áudio de referência. O modelo mostra um bom desempenho em termos de clareza e inteligibilidade, mas com espaço para ajustes, especialmente no que se refere à minimização de distúrbios e à melhoria da qualidade do áudio gerado em ambientes mais exigentes.

3. **Avaliação Subjetiva (MOS):** A métrica MOS foi avaliada por 15 utilizadores, que classificaram os áudios numa escala de 1 (péssimo) a 5 (excelente). O modelo treinado obteve uma pontuação média de 3.9, enquanto o *Google Text-to-Speech* recebeu 4.5. Embora o resultado do modelo treinado tenha sido ligeiramente inferior, a pontuação ainda confirma que a fala gerada é natural, compreensível e bem recebida pelos utilizadores.

5.2 Comparações e Conclusões

Os resultados obtidos demonstram que o sistema de síntese de voz desenvolvido apresenta uma qualidade competitiva quando comparado ao *Google Text-to-Speech*. O áudio gerado pelo modelo treinado foi avaliado positivamente em termos de clareza e inteligibilidade, com métricas como o STOI e MOS indicando um bom desempenho geral. Embora o SNR sugira a necessidade de melhorias na redução de ruídos, o modelo ainda oferece uma qualidade sonora aceitável para aplicações práticas.

Conclui-se que o sistema treinado atende aos objetivos propostos, com um desempenho competitivo em relação ao modelo de referência. No entanto, há espaço para evolução, particularmente na otimização da inteligibilidade e clareza em cenários mais desafiadores, como a redução de ruídos residuais e a melhoria da qualidade da síntese em diferentes condições acústicas.

Segunda Componente do Projeto

6 Modelo de Negócios

O sistema desenvolvido é voltado para o setor da saúde, atendendo hospitais e clínicas que precisam de comunicação acessível e humanizada com pacientes. A principal cliente, Agentifai, procura integrar a tecnologia de conversão de texto em áudio (TTS) para melhorar a interação com os utilizadores.

O produto consiste numa API que converte texto em voz, permitindo personalização de vozes e ajuste de tom para contextos médicos. A API gera áudios naturais, aprimorando a clareza de informações e promovendo acessibilidade para deficientes visuais. Foi implementada em Python, utilizando modelos de *machine learning*, como o *Coqui-TTS*, treinados no *Google Colab*. O sistema opera em *cloud* com tecnologia *Docker*.

O modelo de negócio combina uma estratégia *freemium*, oferecendo funcionalidades básicas gratuitamente, com planos pagos para recursos avançados, como personalização de voz e suporte a múltiplos idiomas. Os planos incluem opções para indivíduos, empresas e um modelo *pay-per-use*, com integração ao *website* e API. Parcerias com instituições educacionais e ONGs também estão previstas.

A entrada no mercado será feita em fases, começando com um piloto gratuito e parcerias estratégicas. A divulgação incluirá *marketing* digital e colaboração com *marketplaces* de APIs. O crescimento será impulsionado por atualizações do produto, como vozes personalizadas, ajuste de emoções no áudio e novos idiomas, além de iniciativas como *hackathons* e colaborações com grandes empresas.

O mercado de TTS em tempo real tem forte potencial de crescimento, projetado para alcançar 9,36 bilhões de dólares em 2031, com grande demanda na América do Norte e Ásia Pacífico. As tendências incluem automatização com IA, acessibilidade e personalização, com concorrentes como *Google Cloud Text-to-Speech*, *Microsoft Azure Speech* e *Amazon Polly*.

Na **Seção Anexo 2** estão representados vários gráficos com informação relativa ao mercado de TTS.

7 Conclusão

No final do desenvolvimento deste trabalho, o grupo está satisfeito com o trabalho que tem vindo a desenvolver ao longo das últimas semanas, tendo conseguido criar um sistema de síntese de voz em tempo real que cumpre os

requisitos que dele foram pedidos. Estes requisitos foram discutidos com os stakeholders tanto durante o seu levantamento como na fase final do projeto de modo a garantir que são adequados ao que o sistema oferece e ao que lhe é requirido, cumprindo este com as especificações e recomendações que nos foram apresentadas pela Agentifai. A nossa abordagem baseada em modelos preexistentes permitiu-nos identificar quais caraterísticas pretendiamos priorizar para o nosso modelo.

7.1 Desafios

Durante a realização do projeto, o nosso grupo enfrentou vários desafios que dificultaram o processo de desenvolvimento, tais como tempo limitado - após a formação de grupos, atribuição de temas e reuniões de grupo (tanto entre os membros como com a empresa) sobraram menos de dois meses para o desenvolvimento do projeto - limites de recursos que abrandaram o progresso do treino de modelos e dificuldade em adquirir certas ferramentas adequadas, nomeadamente *datasets* de voz em PT-PT, já que a vasta maioria dos datasets em português possuem sotaque brasileiro. Contudo, e apesar de preferirmos não ter encontrado algumas destas dificuldades, reconhecemos que é normal estas existirem neste tipo de projeto e que devemos vê-las como aprendizagens para projetos futuros.

7.2 Trabalho Futuro

No contexto deste projeto, o trabalho futuro começa por implementar funcionalidades consideradas relevantes ou úteis que foram idealizadas mas não foram possíveis de incluir no produto final, tais como um ssml-parser (não suportado pelo modelo atual). O *feedback* dos utilizadores finais também é extremamente importante neste aspeto, pois a perspetiva dos utilizadores pode ser vantajosa na identificação de funcionalidades que possam aumentar a satisfação dos consumidores.

7.3 Agradecimentos

O nosso grupo gostaria de agradecer à Agentifai pela proposta do projeto e por se disponibilizar para cobrir custos do *Google Colab Pro*, o que nos permitiu agilizar o treino do modelo e em especial ao João Cunha pela disponibilidade em realizar as sessões de acompanhamento semanais e esclarecer dúvidas a qualquer momento através de email. Também gostariamos de agradecer aos professores João Miguel Fernandes e Victor Alves pela ajuda prestada durante as aulas de acompanhamento à realização do projeto.

A Anexo

Anexo 1 Estrutura do Docker

O sistema foi implementado utilizando **Docker**, sendo dividido em vários *containers* independentes, cada um com um propósito específico. Abaixo, detalhamos a função e configuração de cada container:

1. ***Server*** (`Dockerfile.server`)
   - *Propósito:* Executar o servidor *gRPC*, que processa solicitações de conversas de texto para áudio (TTS).
   - *Configuração:*
     - Baseado na imagem *Python 3.12-slim*.
     - Inclui instalação de dependências como `ffmpeg`, `wget` e pacotes Python necessários.
     - Automatiza o *download* do modelo TTS necessário para execução.
     - Expõe a porta **50051** para comunicação via *gRPC*.
2. ***Client*** (`Dockerfile.client`)
   - *Propósito:* Simular ou executar chamadas ao servidor *gRPC*.
   - *Configuração:*
     - Baseado em Python, configurado para usar a variável de ambiente `SERVER_IP` para especificar dinamicamente o endereço do servidor.
     - Inclui todas as dependências necessárias para a execução do cliente.
3. **API** (`Dockerfile.api`)
   - *Propósito:* Servir como interface intermediária entre o *Sistema* e aplicações externas.
   - *Configuração:*
     - Similar ao cliente, com dependências adicionais específicas para gerenciar comunicações externas.
     - Copia e organiza os ficheiros `.config` e `.proto` necessários.
4. ***Proxy*** (`Dockerfile.proxy`)
   - *Propósito:* Gerir a encaminhamento de requisições e facilitar a comunicação entre os diferentes componentes do sistema.
   - *Configuração:*
     - Atende como ponto de controlo para o fluxo de dados, conectando o cliente, servidor e API.
     - Oferece flexibilidade para escalabilidade.
5. ***Front-end*** (`Dockerfile.app`)
   - *Propósito:* Gerir o ***front-end*** da aplicação, garantindo a interação dos utilizadores com o sistema.
   - *Configuração:*

- Baseado em **Node.js**, com todas as dependências gerenciadas pelo NPM.
- Inclui pacotes específicos para o projeto.
- Expõe a porta configurada na variável de ambiente `PORT` para disponibilizar a interface.

6. ***Normalizer*** (`Dockerfile.normalizer`)
   - *Propósito:* Processar e formatar os textos recebidos antes de o *Server* os processar.
   - *Configuração:*
     - Baseado em Python.
     - Inclui bibliotecas específicas para processamento de texto.
     - Comunica-se diretamente com o Server.

Esboço da Arquitetura dos *Containers Docker*

1. ***Front-end*:** Interface de comunicação com o utilizador. Envia solicitações para a API e recebe as respostas, sejam textos ou áudios.
2. ***Proxy*:** Ponto central de encaminhamento entre *Front-End*, *Client* e API. Gere o fluxo de dados e organiza a comunicação.
3. **API:** Interface intermediária entre o *Front-End* e outros componentes, gere solicitações e conexões externas.
4. ***Client*:** Simula chamadas *gRPC* ao *Proxy* para interagir com o *Server*.
5. ***Server*:** Processa os pedidos de conversão de texto para áudio (TTS), recebendo dados do *Normalizer* ou diretamente do *Proxy*.
6. ***Normalizer:*** Responsável por processar, limpar e formatar os textos antes de serem enviados para o *Server*, garantindo consistência no processamento.

Anexo 2 Análise do Mercado

Em anexo estão vários gráficos e imagens do mercado atual de conversão de texto em voz (TTS) e do seu futuro.

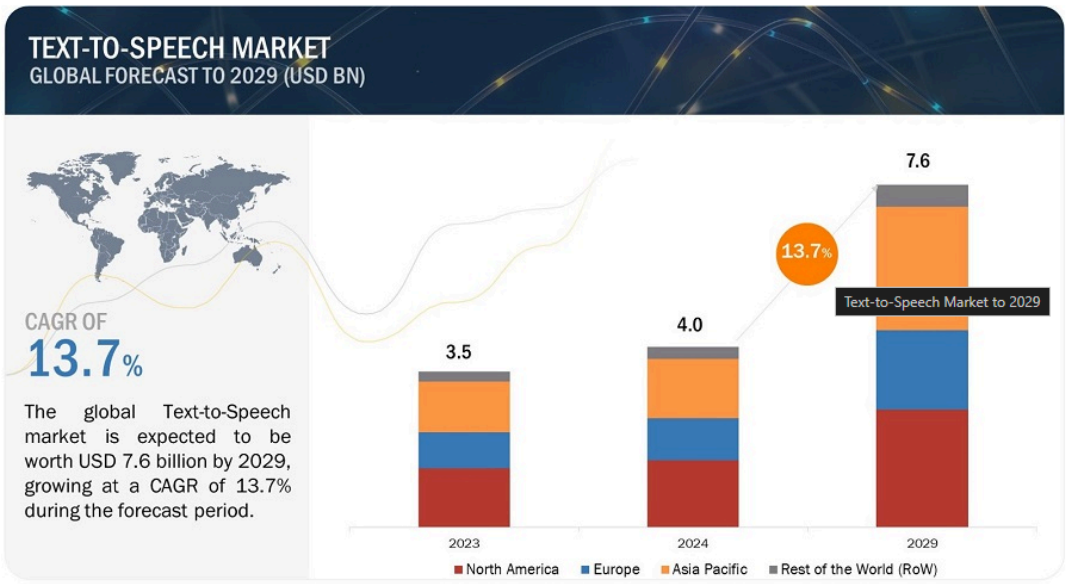


Figura 4: Visão do Mercado TTS



Figura 5: Principais Contribuidores e Consumidores de TTS

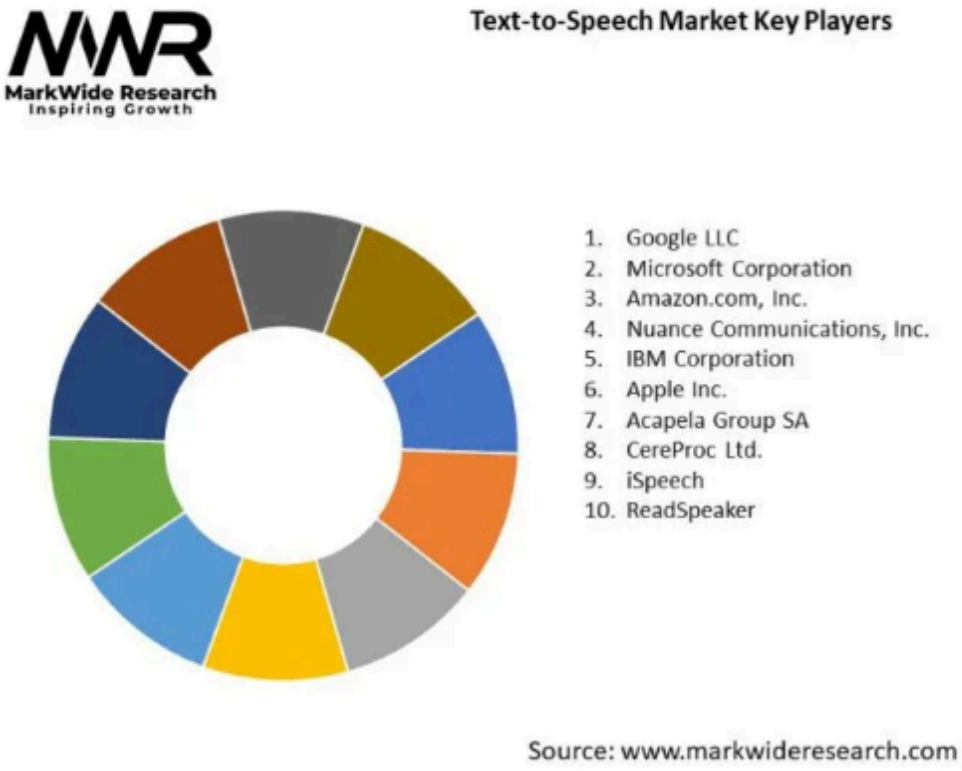


Figura 6: Principais intervenientes no mercado

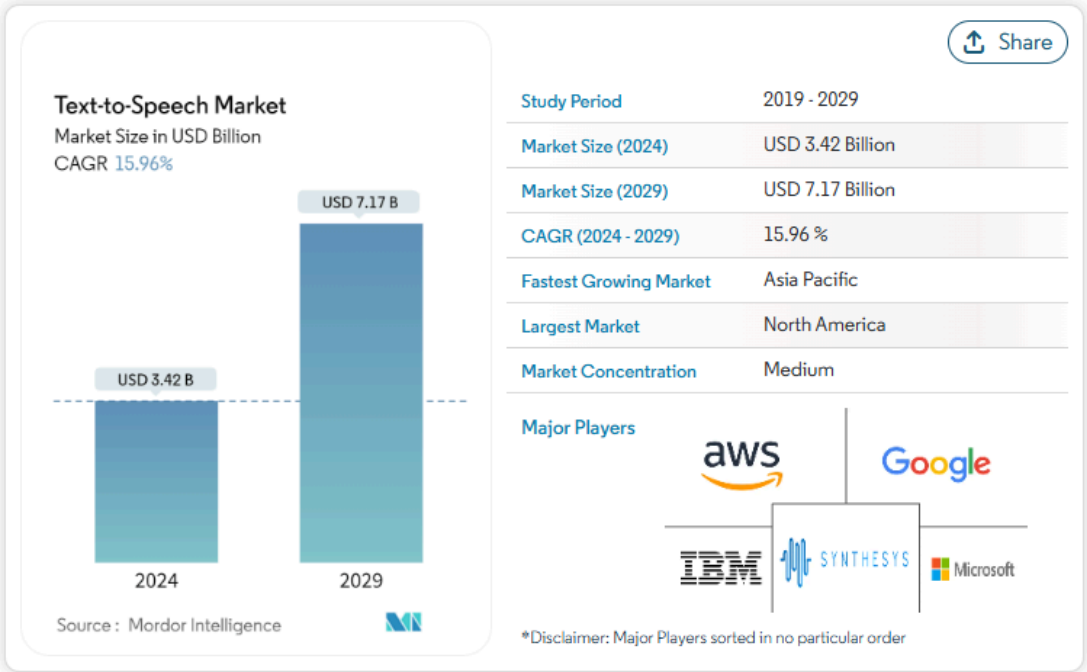


Figura 7: Mercado TTS



Figura 8: Concentração do Mercado TTS